WEEK 9 · GIT

Git & GitHub — 브랜치와 병합

브랜치로 작업을 나누고, merge 로 합치고, 충돌을 해결하고, 실수를 되돌리기

목차

01 브랜치(Branch)와 HEAD 이해하기

02 브랜치 만들기와 전환하기 (git branch · git switch)

03 병합(Merge) — Fast-forward 와 3-way

04 병합 충돌(Conflict) 해결하기

05 브랜치 정리와 원격 브랜치

06 커밋 수정과 되돌리기 (amend · restore · reset · revert)

07 작업 임시 저장하기 (git stash)

08 실습: 브랜치 만들고 충돌 해결하기

1.1

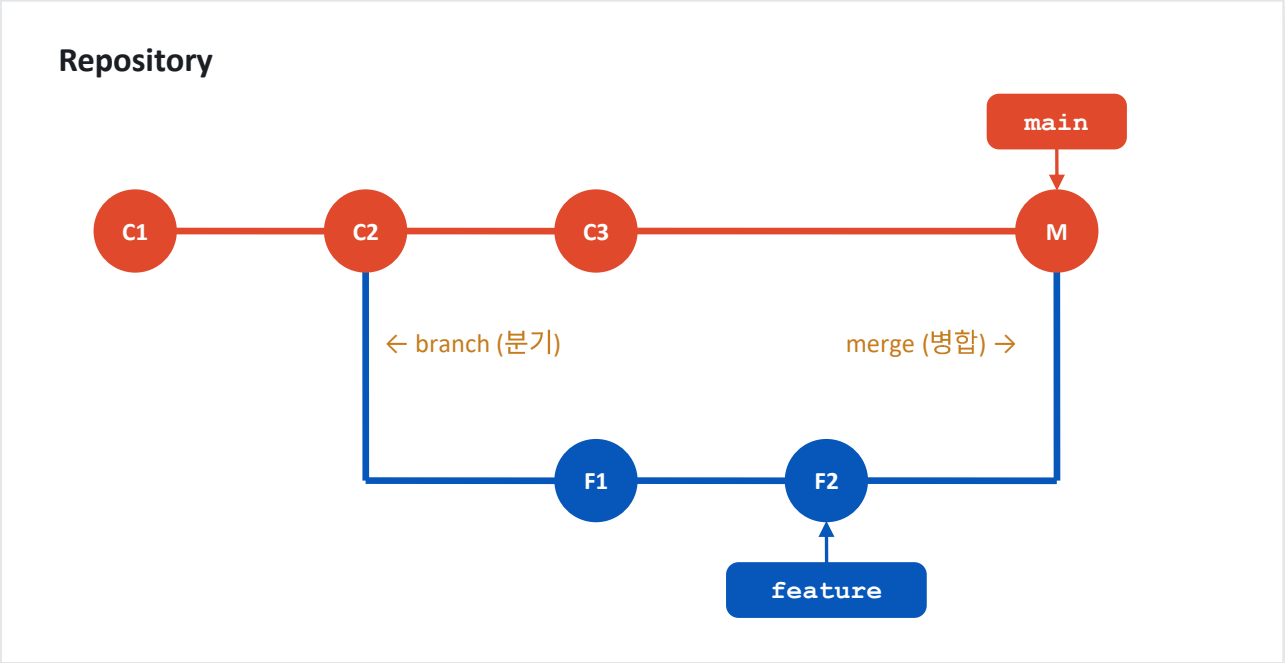
브랜치(Branch)란?

기존 코드에 영향을 주지 않고 독립적으로 작업하기 위한 "작업 흐름"입니다.

1. **격리 (Isolation)**
   새 기능·실험을 main 과 분리해서 개발합니다. 실패해도 main 은 안전합니다.

2. **병렬 작업**
   여러 사람이(또는 한 사람이 여러 작업을) 각자의 브랜치에서 동시에 진행합니다.

3. **병합 (Merge)**
   완성되고 검토된 작업만 main 으로 합칩니다.

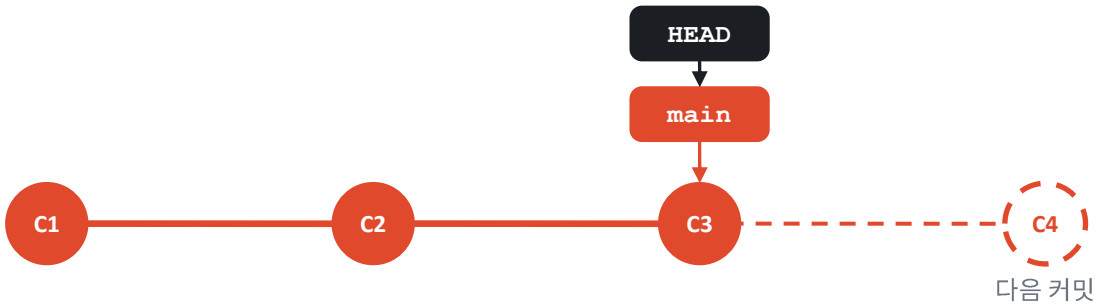


핵심 Git의 브랜치는 파일을 통째로 복사하는 것이 아니라 **커밋 하나를 가리키는 가벼운 포인터**입니다. 그래서 만들고 지우는 비용이 거의 없습니다.

1.2

브랜치와 HEAD — 이름표와 현재 위치

브랜치는 커밋에 붙인 **이름표**, HEAD 는 "지금 내가 있는 브랜치"를 알려주는 **현재 위치 표시**입니다.



새로 커밋하면 main 과 HEAD 가 함께 C4 로 이동합니다

브랜치 (main, devel ...)

- 특정 커밋에 붙은 **이름표**
- 커밋하면 새 커밋으로 자동 이동

HEAD

- 지금 작업 중인 브랜치를 가리킴
- 브랜치를 전환하면 HEAD 도 이동

핵심 새 커밋은 항상 **HEAD 가 가리키는 브랜치**에 쌓입니다. 이것만 기억하면 브랜치가 쉬워집니다.

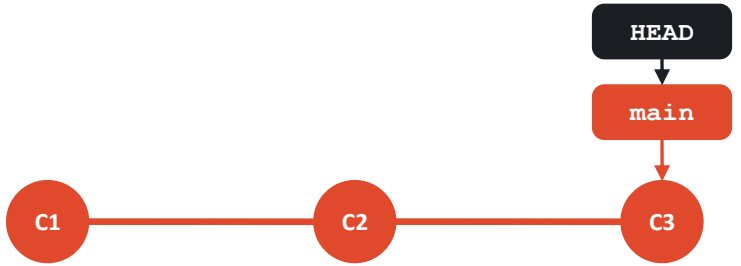
1.3

실습 준비 — 커밋 3개 만들기

새 저장소를 만들고 README.md 에 커밋 3개를 쌓아 브랜치 실습의 출발점을 만듭니다.

branch-lab 저장소 준비

```
$ mkdir branch-lab && cd branch-lab
$ git init
$ echo "My email is me@example.com" > README.md
$ git add README.md
$ git commit -m "Commit 1"
$ echo "Line 2" >> README.md
$ git commit -am "Commit 2"
$ echo "Line 3" >> README.md
$ git commit -am "Commit 3"
$ git log --oneline
44fdf4c (HEAD -> main) Commit 3
51f6827 Commit 2
fbed70e Commit 1
```



-am 옵션 `git commit -am` 은 이미 추적 중인(tracked) 파일의 add 와 commit 을 한 번에 처리합니다. **새 파일(untracked)은 포함되지 않으니** 먼저 `git add` 하세요.

핵심 커밋이 C1 → C2 → C3 순서로 쌓이고, **main 과 HEAD 가 마지막 커밋 C3** 를 가리킵니다.

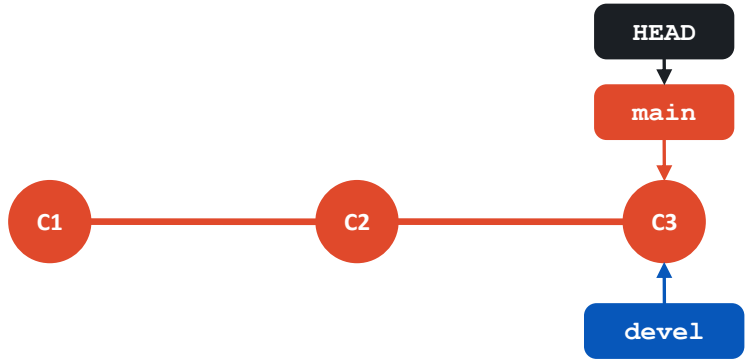
2.1

git branch — 브랜치 만들기 · 목록 보기

`git branch <이름>` 은 현재 커밋을 가리키는 새 브랜치를 만듭니다 (전환은 하지 않음).

git branch

```
$ git branch devel          # 새 브랜치 생성
$ git branch                # 로컬 브랜치 목록
  devel
* main                      # * = 현재 브랜치
```



두 브랜치가 같은 커밋(C3)을 가리킴

핵심 새 브랜치를 만들어도 **HEAD 는 여전히 main** 을 가리킵니다. 브랜치를 만드는 것과 그 브랜치로 옮겨가는 것은 별개의 동작입니다.

2.2

git switch — 브랜치 전환하기

브랜치를 옮겨 다닐 때는 전용 명령 **switch** 를 사용합니다.

브랜치 전환

```
$ git switch devel
Switched to branch 'devel'

# 만들면서 바로 전환
$ git switch -c feature/login
```

하려는 일	예전 명령 (checkout)	이 강의 (switch)
브랜치 전환	`git checkout devel`	`git switch devel`
생성 + 전환	`git checkout -b devel`	`git switch -c devel`

최신 팁 인터넷 자료에 자주 나오는 **checkout** 은 예전 명령입니다. 여전히 동작하지만 하는 일이 많아 헷갈리기 쉬워, 이 강의에서는 **switch** 를 사용합니다.

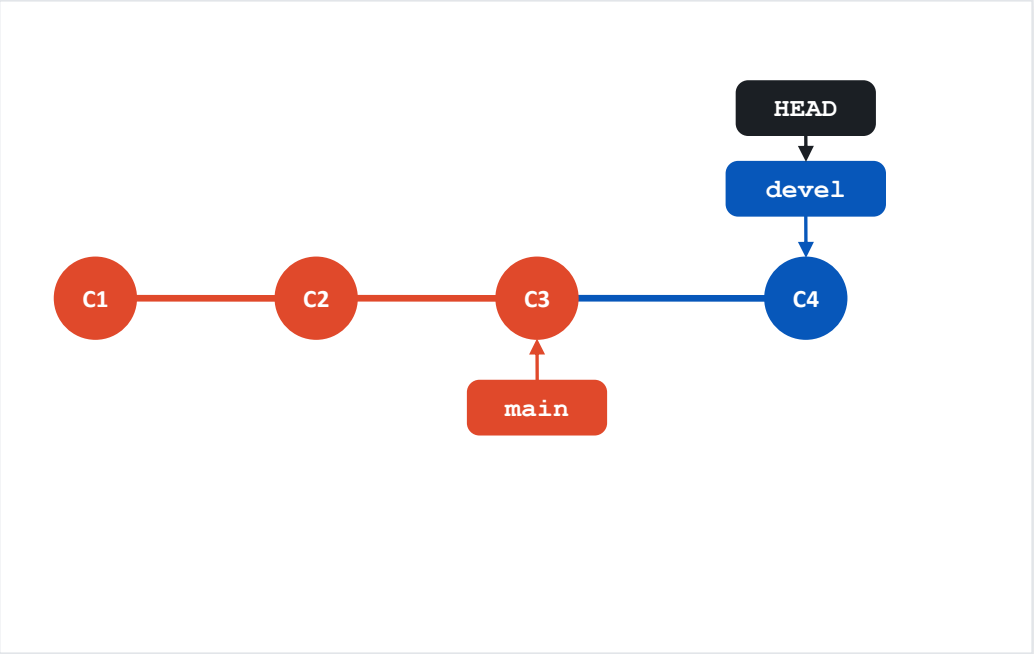
2.3

devel 브랜치에서 작업하고 커밋하기

devel 로 전환한 뒤 새 파일을 추가해 커밋합니다.

devel 에서 커밋

```
$ git switch devel
$ echo "login feature" > login.txt
$ git add login.txt
$ git commit -m "Commit 4: add login"
[devel c9b88d9] Commit 4: add login
$ git status
On branch devel
nothing to commit, working tree clean
```



핵심 커밋은 **HEAD 가 가리키는 브랜치(devel)** 에만 쌓입니다. main 은 Commit 3 에 그대로 머물러 있습니다.

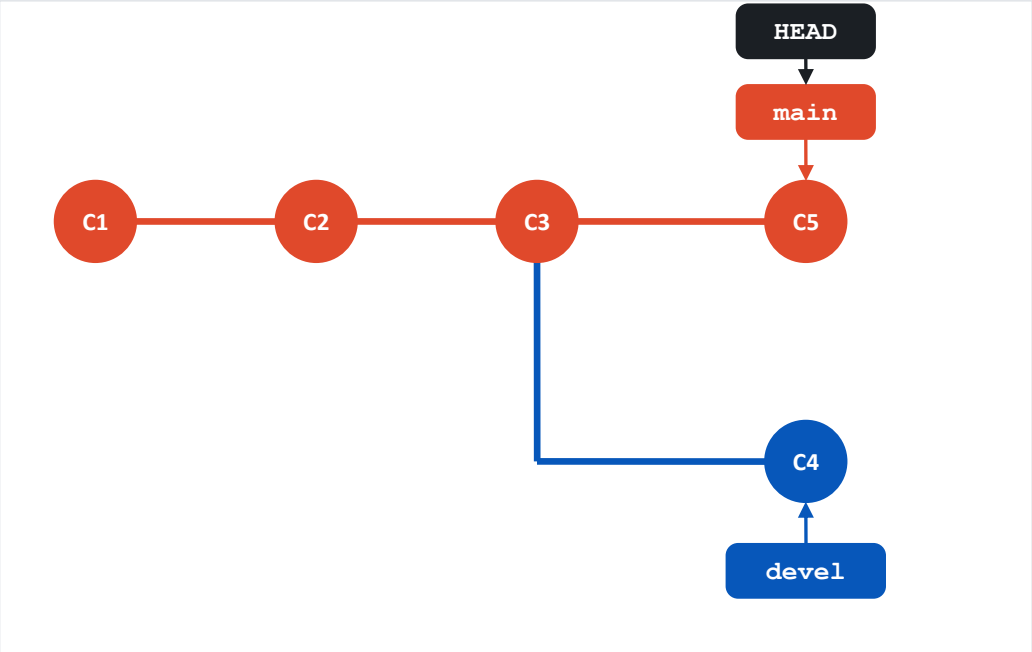
2.4

main 으로 돌아와 커밋하기 — 이력이 갈라짐

브랜치를 전환하면 작업 디렉터리의 파일도 그 브랜치의 최신 커밋 상태로 바뀝니다.

main 에서 커밋

```
$ git switch main
$ ls
README.md                # login.txt 가 없음!
$ echo "Line 4 (main)" >> README.md
$ git commit -am "Commit 5"
[main 6464eb8] Commit 5
```



주의 커밋하지 않은 수정이 있는 채로 전환하면, 전환이 거부되거나 수정 내용이 다른 브랜치로 따라옵니다. 전환 전에 **커밋**하거나 `git stash`(7장)로 **임시 저장**하세요.

2.5

브랜치 이력을 그래프로 보기

`git log --oneline --graph --all` 로 모든 브랜치의 흐름을 터미널에서 한눈에 확인합니다.

git log --graph

```
$ git log --oneline --graph --all
* 6464eb8 (HEAD -> main) Commit 5
| * c9b88d9 (devel) Commit 4: add login
|/
* 44fdf4c Commit 3
* 51f6827 Commit 2
* fbed70e Commit 1
```

그래프 읽는 법

- `*` 커밋 하나
- `|` 브랜치의 흐름(선)
- `|/` 브랜치가 갈라진 지점
- **`(HEAD -> main)`** 현재 위치한 브랜치
- **`(devel)`** 그 커밋을 가리키는 다른 브랜치

최신 팁 VS Code 의 Source Control 그래프, GitHub Desktop 같은 GUI 도구에서도 같은 그래프를 볼 수 있습니다. 명령어 출력과 비교하며 익혀 보세요.

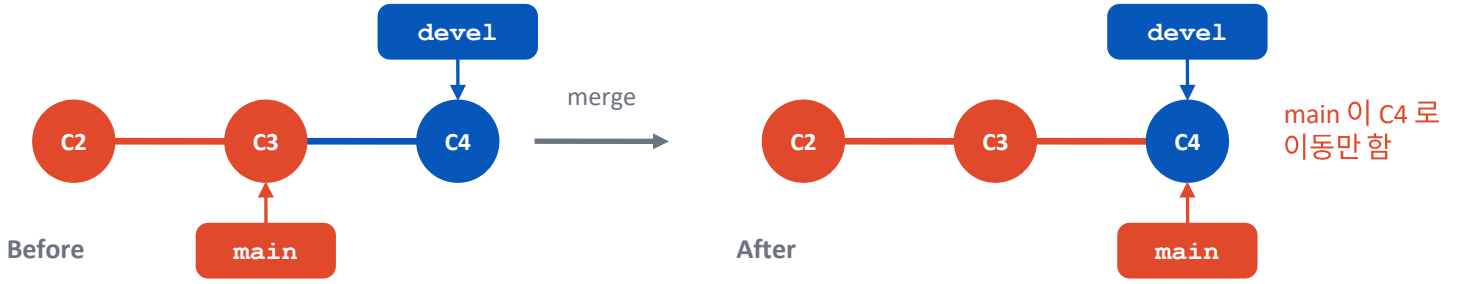
3.1

병합(Merge)의 두 가지 방식

`git merge devel` 은 "devel 의 변경을 현재 브랜치(main)로 합친다"는 뜻입니다. 이력 모양에 따라 방식이 달라집니다.

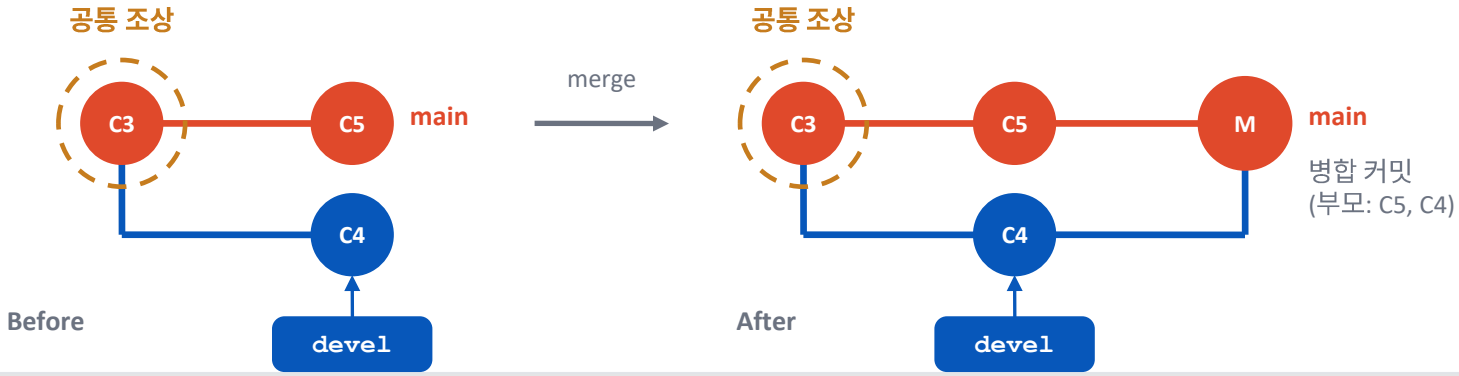
① Fast-forward 병합

이력이 **일직선**일 때
main 포인터만 앞으로 이동
새 커밋이 생기지 않음



② 3-way 병합

이력이 **갈라졌을** 때
공통 조상(C3) · main 끝(C5) · devel 끝(C4)
세 커밋을 비교해 합친 뒤
병합 커밋 M 을 새로 만듦



핵심 **공통 조상** = 두 브랜치가 갈라지기 직전의 마지막 공통 커밋(C3). 병합 전에는 받는 쪽으로 먼저 **`git switch main`** 하세요.

3.2

Fast-forward 병합 vs --no-ff

main 에는 새 커밋이 없고 devel 만 앞서 나간 경우(일직선), 기본 병합은 main 이름표만 앞으로 옮깁니다.

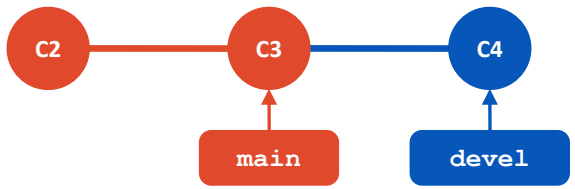
기본: fast-forward

```
$ git switch main
$ git merge devel
Updating 44fdf4c..c9b88d9
Fast-forward                    # 새 커밋 없음
```

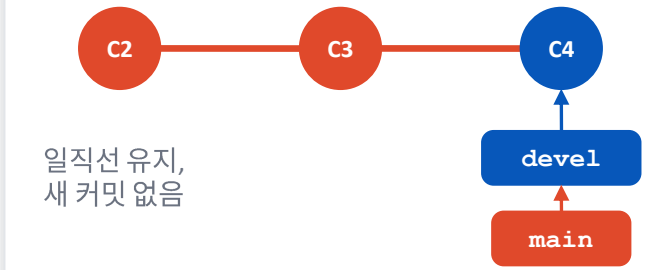
--no-ff: 병합 커밋을 강제로 생성

```
$ git switch main
$ git merge --no-ff devel
Merge made by the 'ort' strategy.
# 커밋 메시지 편집기가 열리면 저장 후 종료
```

Before



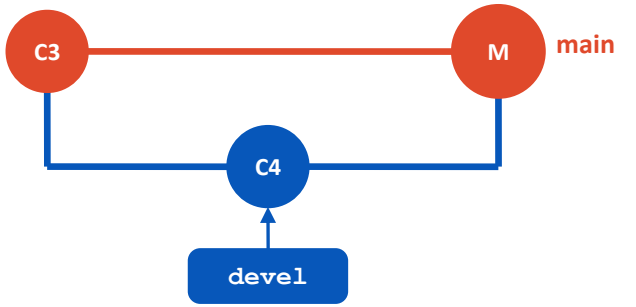
After — 기본 (fast-forward)



일직선 유지,
새 커밋 없음

After — --no-ff

새 병합 커밋



병합 커밋 M C4 가 아니라 **새로 생긴 커밋**입니다. 부모가 2개(C3, C4)이고 파일 내용은 C4 와 같아, "devel 을 여기서 병합했다"는 흔적만 이력에 남깁니다. GitHub PR 의 기본 병합도 이 방식입니다.

3.3

3-way 병합 — 병합 커밋 만들기

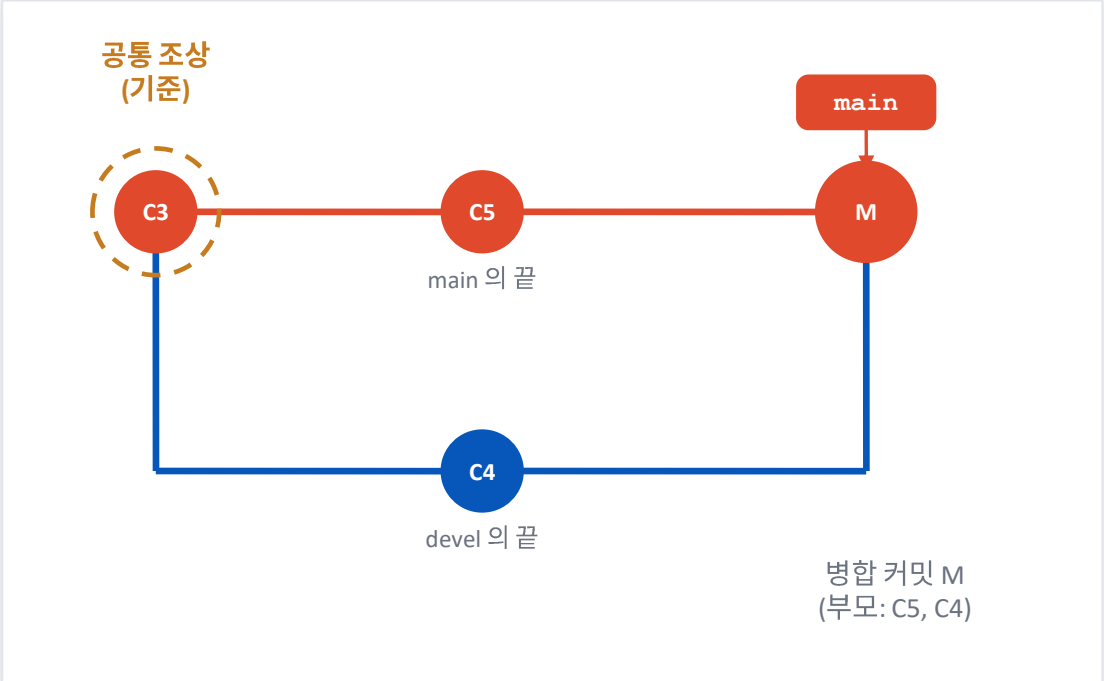
합치는 대상은 **C5(main)와 C4(devel)** 입니다. 공통 조상 C3 는 "각자 무엇을 바꿨는지" 알아내는 **기준**입니다.

3-way merge

```
$ git switch main
$ git merge devel
Merge made by the 'ort' strategy.
 login.txt | 1 +
```

Git 의 판단 방법 (README 의 한 줄을 예로)

C3 (기준)	C5 (main)	C4 (devel)	병합 결과
A	**B**	A	main 만 바꿈 → **B**
A	A	**C**	devel 만 바꿈 → **C**
A	**B**	**C**	둘 다 바꿈 → **충돌!**

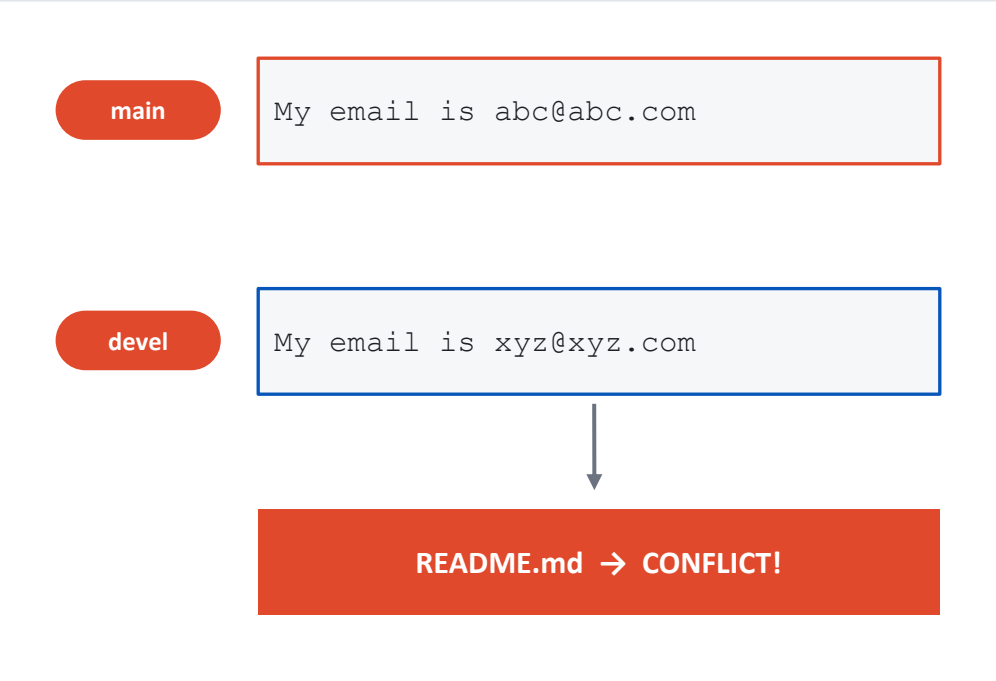


핵심 한쪽만 바꾼 부분은 Git 이 **자동으로** 합칩니다. 같은 줄을 양쪽이 다르게 바꾼 경우에만 **충돌**이 납니다 (4장).

4.1

병합 충돌(Merge Conflict)은 언제 생기나?

두 브랜치가 **같은 파일의 같은 줄**을 서로 다르게 고치면, Git 은 어느 쪽이 맞는지 스스로 판단할 수 없습니다.



충돌 발생

```
$ git switch main
$ git merge devel
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
$ git status
You have unmerged paths.      # 병합이 멈춘 상태
Unmerged paths:
        both modified:   README.md
```

핵심 충돌은 오류가 아니라 **"사람의 판단이 필요하다"는 신호**입니다. Git 은 병합을 일시 정지하고 여러분이 해결할 때까지 기다립니다.

4.2

충돌 마커(Conflict Marker) 읽는 법

Git 은 충돌한 파일 안에 양쪽 내용을 모두 넣고, 마커로 구역을 표시해 둡니다.

README.md (충돌 상태)

```
<<<<<<< HEAD
This is the README. My email is abc@abc.com
=======
This is the README. My email is xyz@xyz.com
>>>>>>> devel
```

`<<<<<<< HEAD ~ =======`
현재 브랜치(main, 받는 쪽)의 내용
`======= ~ >>>>>>> devel`
병합하려는 브랜치(devel)의 내용

해결 방법은 세 가지 중 하나

1. **내 것 유지**
   abc@abc.com 만 남김

2. **상대 것 채택**
   xyz@xyz.com 만 남김

3. **직접 조합**
   두 내용을 합쳐 새로 작성

주의 마커 줄(<<<<<<<, =======, >>>>>>>)을 **하나도 남기지 말고 모두 지워야** 합니다. 남긴 채 커밋하면 그대로 코드에 들어가 빌드 오류가 납니다.

4.3

충돌 해결 4단계

확인 → 수정 → 표시 → 완료. 해결이 어렵다면 언제든 병합 이전 상태로 되돌릴 수 있습니다.

1 확인

`git status`

충돌 파일 찾기

2 수정

파일 편집

마커 삭제·내용 결정

3 표시

`git add <file>`

"해결했음"을 알림

4 완료

`git commit`

병합 커밋 생성

충돌 해결 / 취소

```
$ vi README.md              # 원하는 내용으로 고치고 마커 삭제
$ git add README.md         # 해결 완료 표시
$ git commit                # 병합 메시지가 자동으로 채워짐

# 병합을 포기하고 병합 전 상태로 돌아가기
$ git merge --abort
```

최신 팁 VS Code 는 충돌 구역 위에 **Accept Current / Accept Incoming / Accept Both** 버튼과 3-way **Merge Editor** 를 제공해, 클릭만으로 해결할 수 있습니다.

핵심 `git add` 는 "스테이징"이면서 동시에 **"이 파일의 충돌을 해결했다"는 표시**입니다. 모든 충돌 파일을 add 해야 커밋할 수 있습니다.

5.1

병합된 브랜치 정리하기

역할을 다한 브랜치는 지워서 목록을 깔끔하게 유지합니다.

git branch -d

```
$ git branch --merged       # 현재 브랜치에 병합 완료된 목록
  devel
* main
$ git branch -d devel
Deleted branch devel (was c9b88d9).

$ git branch -d experiment
error: the branch 'experiment' is not fully merged.
$ git branch -D experiment  # 강제 삭제 (주의!)
```

-d vs -D

옵션	동작
`-d`	안전 삭제 — 병합되지 않은 브랜치는 거부
`-D`	강제 삭제 — 병합 여부와 상관없이 삭제

`git branch --no-merged`로 아직 병합되지 않은 브랜치를 먼저 확인하면, 지우면 안 되는 브랜치를 알 수 있습니다.

핵심 브랜치를 지워도 **이미 병합된 커밋은 main 이력에 그대로 남습니다.** 지워지는 것은 "포인터(이름표)" 뿐입니다.

5.2

브랜치도 GitHub 에 올릴 수 있다

지금까지는 main 만 push 했지만, 다른 브랜치도 같은 방법으로 올리고 받을 수 있습니다.

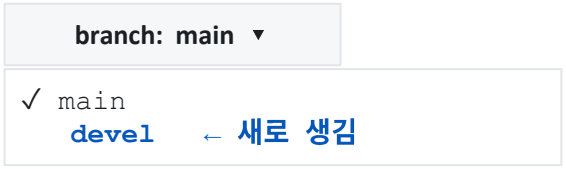
1 올리기 — Alice

main 을 올릴 때와 똑같은 명령, **브랜치 이름만** 바뀝니다.

```
$ git push -u origin devel
```

2 확인 — GitHub

저장소 화면의 브랜치 메뉴에 devel 이 새로 보입니다.



3 받기 — Bob

fetch 로 GitHub 의 새 브랜치 소식을 받고, switch 로 devel 에 들어갑니다.

```
$ git fetch
$ git switch devel
```

혼자 작업할 때

보통 **main 만 push** 해도 충분합니다.
브랜치는 내 PC 안에서만 쓰고, 병합한 뒤 main 을 올립니다.

팀으로 작업할 때

기능 브랜치를 push 하면 동료가 확인 · 리뷰할 수 있습니다.
→ 다음 시간 Pull Request 의 출발점입니다.

삭제 GitHub 의 브랜치를 지우려면 `git push origin --delete devel` (내 PC 의 devel 은 그대로 남습니다).

6.1

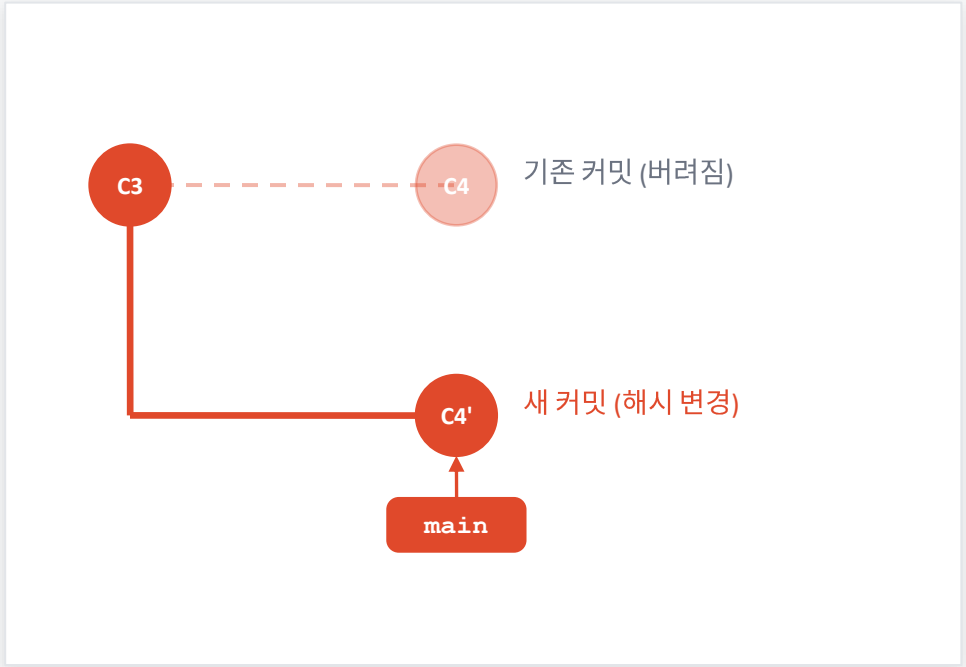
git commit --amend — 직전 커밋 고치기

커밋 메시지에 오타가 있거나 파일을 빠뜨렸을 때, 새 커밋을 만들지 않고 직전 커밋을 고칩니다.

amend

```
$ git commit -m "added login menu"
# 앗, login.css 를 빠뜨렸다!
$ git add login.css
$ git commit --amend -m "Add login menu with styles"

# 메시지는 그대로 두고 파일만 추가
$ git commit --amend --no-edit
```



주의 amend 는 기존 커밋을 **해시가 다른 새 커밋으로 교체**합니다. 이미 push 해서 공유한 커밋에는 사용하지 마세요 — 동료의 이력과 어긋나 push 가 거부됩니다.

6.2

되돌리기 명령 한눈에 보기

"무엇을 되돌리고 싶은가"에 따라 사용할 명령이 다릅니다.

상황	명령어	결과
작업 디렉터리의 수정을 취소	`git restore <file>`	마지막 커밋 상태로 복원 (수정 내용 삭제, 복구 불가)
스테이징만 취소	`git restore --staged <file>`	수정 내용은 그대로 유지
직전 커밋의 메시지·파일 수정	`git commit --amend`	직전 커밋을 새 커밋으로 교체
로컬 커밋 취소 (이력 삭제)	`git reset <commit>`	브랜치 포인터를 과거 커밋으로 이동
공유된 커밋 취소 (이력 유지)	`git revert <commit>`	변경을 반대로 적용한 새 커밋 추가

판단 기준 **"이미 push 해서 다른 사람과 공유했는가?"** 공유했다면 → **`revert`**. 아직 내 컴퓨터에만 있다면 → **`amend`** / **`reset`** 도 괜찮습니다.

6.3

git reset 의 세 가지 모드

`git reset <commit>` 은 현재 브랜치를 지정한 커밋으로 되돌립니다. 모드에 따라 어디까지 되돌릴지가 달라집니다.

	작업 디렉터리	스테이징 영역	브랜치 (HEAD)	
`--soft`	유지	유지	**되돌림**	커밋만 취소. 변경 내용은 staged 상태로 남음 → 커밋 다시 하기
`--mixed` (기본값)	유지	**되돌림**	**되돌림**	커밋 + 스테이징 취소. 파일 수정 내용은 남음
`--hard`	**되돌림**	**되돌림**	**되돌림**	모두 취소. 수정한 내용까지 삭제됨 ⚠

reset

```
$ git reset --soft HEAD~1      # HEAD~1 = 한 커밋 전
$ git reset HEAD~2             # --mixed, 두 커밋 전으로
```

주의 **`--hard`** 는 커밋하지 않은 수정 내용을 **영구히 삭제**합니다. 실행 전에 **`git status`** 로 꼭 확인하세요.

핵심 reset 은 커밋을 "지우는" 것처럼 보이지만, 실제로는 **브랜치 포인터를 과거로 옮기는 것**입니다. 처음에는 수정 내용이 남는 **`--soft`** 부터 연습하세요.

6.4

git revert — 이력을 남기며 되돌리기

실수한 커밋을 지우는 대신, 그 커밋을 **반대로 고친 새 커밋**을 하나 더 만듭니다.

아래 상자 = 각 커밋 시점의 README.md 내용



→ C4 의 파일 내용은 C2 와 같아지지만, **C3 는 이력에서 지워지지 않고 그대로 남습니다.**

revert

```
$ git revert HEAD            # 가장 최근 커밋(C3)을 되돌림
[main 7a1d2e3] Revert "Commit 3"
$ git log --oneline
7a1d2e3 (HEAD -> main) Revert "Commit 3"
44fdf4c Commit 3             # 그대로 남아 있음
```

핵심 revert 는 커밋을 **지우지 않고**, 반대로 고친 새 커밋을 **추가**합니다. 이력이 바뀌지 않으므로 이미 push 한 main 에서도 안전하게 쓸 수 있습니다.

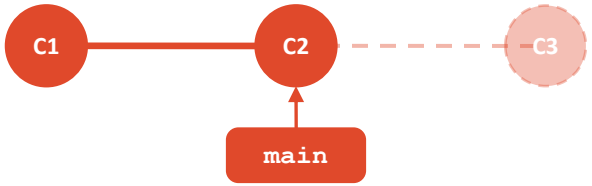
6.5

reset vs revert

같은 "되돌리기"지만 이력을 다루는 방식이 정반대입니다.

git reset

이력을 지움 (과거로 이동)



- 아직 push 하지 않은 내 커밋을 정리할 때
- 커밋이 이력에서 사라짐
- 공유한 뒤에 쓰면 동료의 이력과 어긋남

git revert

이력을 추가 (반대 커밋)



- 이미 push 한 커밋을 되돌릴 때
- 되돌린 기록이 새 커밋으로 남음
- 동료에게 안전

기억하기 **push 하기 전**이면 reset, **push 한 후**면 revert. 공유된 이력은 지우지 말고, 되돌리는 커밋을 덧붙입니다.

7.1

git stash — 하던 작업 잠시 치워두기

"긴급 버그 수정 요청!" — 그런데 지금 작업은 커밋하기엔 아직 미완성일 때 사용합니다.

작업 중
devel 에서
로그인 화면 수정

→

stash
수정 내용을
임시 보관함에

→

긴급 수정
main 으로 전환
버그 수정 & 커밋

→

stash pop
devel 로 돌아와
작업 내용 복원

stash

```
$ git stash                    # 변경 내용 보관
$ git switch main              # 깨끗한 상태로 전환
#    ... 긴급 수정 후 커밋 ...
$ git switch devel             # 원래 브랜치로 복귀
$ git stash list               # 보관함 목록 확인
stash@{0}: WIP on devel: ...
$ git stash pop                # 꺼내서 적용 + 목록에서 제거
```

기억할 명령 3개

`git stash` 하던 작업 치워두기

`git stash list` 보관 목록 보기

`git stash pop` 다시 꺼내기

최신 팁 메시지를 붙일 때는 **`git stash push -m "..."`** 를 사용하세요.

08

실습: 브랜치 만들고 충돌 해결하기

branch-lab 저장소에서 아래 순서대로 진행하고, 단계마다 **`git log --oneline --graph --all`** 로 그래프를 확인하세요.

단계	실습 내용
준비	branch-lab 저장소 생성 → README.md 로 Commit 1~3 작성 (1.3)
브랜치	**`git switch -c devel`** → login.txt 추가 → Commit 4
분기	**`git switch main`** → README.md 수정 → Commit 5 → 그래프에서 갈라진 이력 확인
3-way 병합	main 에서 **`git merge devel`** → 병합 커밋(부모 2개) 확인
충돌 만들기	main·devel 각각에서 README.md 의 이메일 줄을 다르게 수정하고 커밋
충돌 해결	**`git merge devel`** → 마커 수정 → **`git add`** → **`git commit`** (한 번은 **`git merge --abort`** 도 체험)
되돌리기 (revert)	**`git revert HEAD`** 로 마지막 커밋 취소 → **`git log --oneline`** 으로 Revert 커밋이 추가된 것 확인
되돌리기 (reset)	새로 커밋 하나 → **`git reset --soft HEAD~1`** 로 커밋만 취소 → **`git status`** 로 staged 상태 확인
임시 저장	devel 에서 README.md 수정 중 **`git stash`** → **`git switch main`** → 다시 **`git switch devel`** → **`git stash pop`**
정리 & 공유	**`git branch -d devel`** → GitHub 저장소 연결 후 **`git push -u origin main`**

정리하기

1. 브랜치는 커밋을 가리키는 가벼운 포인터이고, HEAD 는 지금 작업 중인 브랜치를 가리킵니다.
2. `git switch -c` 로 만들고 전환하며, `git log --oneline --graph --all` 로 흐름을 확인합니다.
3. `git merge` 는 일직선이면 fast-forward, 갈라졌으면 3-way 병합(병합 커밋)으로 합칩니다.
4. 충돌은 마커 수정 → `git add` → `git commit` 으로 해결하고, 포기할 땐 `git merge --abort`.
5. 공유 전 커밋은 amend / reset, 공유된 커밋은 revert 로 되돌리고, 하던 작업은 stash 로 임시 보관합니다.
6. 다음 시간에는 fetch · pull, Pull Request, 협업 워크플로우(GitHub Flow)를 다룹니다.